GrapeMaster: An End-to-End Mobile Decision-Support Platform for Vineyard Disease Warning and Precision Management

Author information to be added

Abstract

Grapevine disease management requires timely integration of weather data, crop phenology, disease-risk models, cultivar susceptibility, fungicide records, and field observations. GrapeMaster is an end-to-end software platform designed to connect these data sources with mobile vineyard operations. The current system combines a Flutter mobile application, a Django REST Framework backend, PostgreSQL storage, scheduled weather and model workflows, a FastAPI disease-model service, image-recognition modules, notification services, and field-task management. This manuscript draft documents the system architecture as the foundation for a full platform paper. Future sections will report deployment scale, model validation, image-recognition performance, and user-workflow evaluation.

Keywords: precision viticulture; grapevine disease warning; decision support; phenology model; disease-risk model; mobile agriculture; Django; Flutter; FastAPI

1 Introduction

Plant pathogens and pests remain a major constraint on agricultural production, reducing both yield and quality and creating strong demand for timely, site-specific crop-health management [Savary et al., 2019]. Grapevine production is particularly exposed to disease risks because major foliar and fruit diseases, including downy mildew, powdery mildew, gray mold, and black rot, respond dynamically to local weather, host growth stage, cultivar susceptibility, canopy condition, and recent plant-protection operations [Peng et al., 2024, Kunova et al., 2021, Rahman et al., 2024, Szabó et al., 2023, Puelles et al., 2024]. Gray mold and other bunch rots affect grape and wine quality as well as yield, while powdery mildew and downy mildew management remains closely linked to fungicide timing and resistance-aware disease control [Steel et al., 2013, Kunova et al., 2021, Peng et al., 2024]. In practice, disease control is also shaped by environmental and regulatory pressure to reduce unnecessary pesticide inputs while maintaining fruit quality and yield stability [Mian et al., 2021, Rossi et al., 2014]. Disease management in vineyards is therefore not only a diagnostic problem, but a continuous decision process in which growers must interpret infection-conducive weather, phenological susceptibility, visible symptoms, treatment windows, and fungicide protection history within a short operational period [Rosa et al., 1993, Rossi et al., 2014, Puelles et al., 2024].

Several technical approaches have been developed to support this process, especially weather-driven disease models, phenological models, decision-support systems, and image-based diagnosis methods [Rosa et al., 1993, Rossi et al., 2014, Ortega-Farias and Riveros-Burgos, 2019, Xie et al., 2020]. Weather-driven disease models have long been used to simulate pathogen development and guide fungicide timing in vineyards [Rosa et al., 1993, Orlandini et al., 1995]. For example, PLASMO simulated grapevine downy mildew development from temperature, leaf wetness, relative humidity, and treatment information, and was designed for direct use in vineyard management [Rosa et al., 1993]. Later model-based systems and evaluations showed that downy mildew prediction remains sensitive to regional climatic conditions and therefore requires field-level calibration, validation, and operational interpretation rather than simple transfer of generic

infection rules [Orlandini et al., 1995, Puelles et al., 2024]. Vineyard decision-support systems such as vite.net combined real-time monitoring, data storage, mechanistic models, alerts, and web-based decision support, explicitly addressing the implementation problem that has limited the sustained use of many agricultural DSSs [Rossi et al., 2014, McCown, 2002, Rose et al., 2016]. In parallel, grapevine growth-stage scales and phenology models provide a formal basis for linking crop development to management timing, including BBCH or modified Eichhorn-Lorenz stage descriptions and cumulative-degree-day model formulations [Lorenz et al., 1995, Coombe, 1995, Ortega-Farias and Riveros-Burgos, 2019, Molitor et al., 2014]. Recent cultivar-scale and multi-stage phenology models further indicate that temperature-driven development can be represented quantitatively across cultivars, although calibration data and regional context affect model behavior [Caffarra et al., 2014, Molitor et al., 2020, Ortega-Farias and Riveros-Burgos, 2019].

Recent advances in computer vision have added another important source of field evidence for crop-health assessment [Xie et al., 2020, Liu et al., 2020, Shi et al., 2023]. Deep convolutional networks, vision transformers, and object-detection models have been applied to grape leaf disease classification, downy mildew detection, and lesion localization under natural or semi-natural imaging conditions [Xie et al., 2020, Liu et al., 2020, Zhang et al., 2022, Kunduracioglu and Pacal, 2024, Liu et al., 2024]. Disease-severity studies further emphasize the value of pixel-level lesion and leaf segmentation for quantifying symptom intensity instead of reporting only a class label [Shi et al., 2023]. These methods can reduce dependence on manual visual inspection and support fast symptom-level assessment in the field [Xie et al., 2020, Zhang et al., 2022, Kunduracioglu and Pacal, 2024]. However, image diagnosis alone does not explain whether weather and phenology are favorable for infection, whether a treatment window is open, or whether previous fungicide applications still provide protection, because those decisions depend on weather-driven epidemiology, growth stage, and treatment history [Rosa et al., 1993, Rossi et al., 2014, Puelles et al., 2024].

The main gap is therefore one of system integration, because prior work often advances disease forecasting, phenology simulation, image diagnosis, or farm recording as separate technical functions [Rossi et al., 2014, Fountas et al., 2015, Tummers et al., 2019, Kunduracioglu and Pacal, 2024]. Farm management information systems have evolved from record-keeping tools toward complex decision-support environments, yet reviews continue to report persistent obstacles such as data fragmentation, lack of interoperability, usability constraints, and insufficient connection between analytical functions and farm operations [Fountas et al., 2015, Tummers et al., 2019, Rose et al., 2016]. Agricultural DSS research has also emphasized that model value depends on delivery, usability, trust, and fit with decision contexts rather than on predictive algorithms alone [McCown, 2002, Rose et al., 2016]. A grower may receive a risk index in one tool, record spraying in another tool, and inspect disease symptoms through a separate image-recognition workflow, mirroring the broader fragmentation described for agricultural information systems [Fountas et al., 2015, Tummers et al., 2019]. This separation limits the ability to convert model outputs into field tasks and to feed completed operations back into later risk calculations, a challenge that vineyard DSS work has tried to address through closer coupling between monitoring, models, warnings, and management advice [Rossi et al., 2014, Mian et al., 2021]. End-to-end agricultural platforms have shown the value of integrating data acquisition, ingestion, storage, analytics, and user-facing decision support into one scalable workflow; a similar platform perspective is needed for vineyard disease warning and mobile field management [Bakir et al., 2026, Rossi et al., 2014, Fountas et al., 2015].

GrapeMaster was developed to address this gap through a crop-season-centered mobile decision-support platform for vineyard disease warning and precision management. The platform links vineyard fields, crop seasons, weather retrieval, phenology simulation, disease-risk modeling, spraying-suitability assessment, fungicide task feedback, image-based disease assistance, field notes, notifications, and grower communication. Its key design principle is a closed manage-

ment loop: weather and crop-season data feed model workflows; model outputs are displayed as warnings, risk summaries, treatment windows, and recommendations; users create and complete plant-protection tasks; and completed tasks are converted into fungicide-history inputs for subsequent disease-risk recalculation.

This paper presents GrapeMaster as an end-to-end mobile software architecture for vineyard disease warning and precision management. The contribution is not a single isolated model, but the integration of field- and crop-season-centered data management, weather-driven phenology simulation, disease-risk modeling, spraying-suitability assessment, image-based disease assistance, notifications, plant-protection task execution, and fungicide-history feedback. By connecting disease warnings, treatment windows, task records, and completed fungicide applications into one workflow, the platform exposes analytical results to growers through mobile interfaces while preserving the operational data needed for subsequent model recalculation and management review.

2 System Architecture

GrapeMaster uses a layered architecture in which the mobile client, backend services, analytical model services, persistent storage, and external data services are separated but connected through REST APIs, scheduled jobs, WebSocket channels, and database records. The central design unit is the crop season. A crop season links a vineyard field with cultivar information, cultivation method, vine age, weather history, phenology outputs, disease-risk simulations, fungicide applications, tasks, notes, and notifications. This section summarizes the architecture in two parts: conceptual architecture and logical architecture.

The platform is implemented with a mixed programming stack selected according to layer responsibilities. The mobile application is written in Dart with Flutter, with an Android build shell configured through Kotlin and Gradle on the Java-compatible Android toolchain. The backend business service and analytical workflows are written in Python and organized as a Django 4.2 project with Django REST Framework, Django Channels, Django APScheduler, SimpleJWT, and PostgreSQL access through `psycopg2`. The independent disease-model service is also written in Python and exposed through FastAPI, Uvicorn, Pydantic, and Starlette. Numerical and time-series computation relies mainly on NumPy, Pandas, Python Dateutil, and Pytz. Geospatial and weather-related processing uses Shapely, Fiona, GeoPandas, Geopy, Requests, HTTPX, Open-Meteo client packages, request caching, and retry utilities. Image recognition and disease-severity estimation use Python deep-learning and image-processing packages, including PyTorch, TorchVision, Ultralytics YOLO/SAM components, EfficientViT modules, OpenCV, Pillow, and NumPy.

2.1 Conceptual Architecture

Conceptually, GrapeMaster implements a closed management loop from field data acquisition to disease-risk interpretation and back to field operations. A user first creates a vineyard field and crop season in the mobile application. The backend stores the field geometry, centroid, crop metadata, cultivar susceptibility, and cultivation method. Scheduled backend jobs retrieve historical and forecast weather for the field centroid. Weather records are transformed into inputs for phenology simulation, spraying-suitability assessment, and disease-risk modeling. Disease-model outputs are stored in the backend, exposed to the mobile application, and translated into warnings, treatment windows, recommendations, and task-related decisions.

Figure 1 summarizes the main system flow.

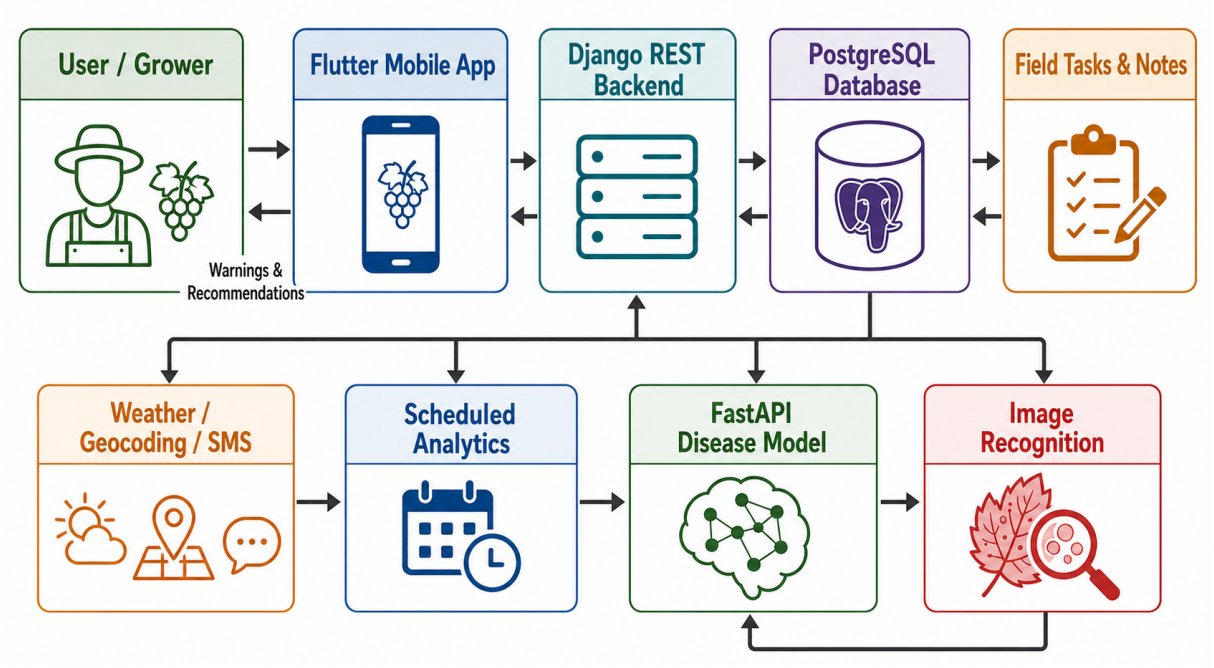

Figure 1: Conceptual architecture of the GrapeMaster platform. The platform connects mobile field operations, backend business services, relational storage, scheduled analytical workflows, disease-risk simulation, image-analysis modules, and external data services.

2.2 Logical Architecture

The logical architecture follows the same five-part structure used in the implementation section. Rather than treating the platform as a set of isolated modules, GrapeMaster organizes the application, data, analytical models, image diagnosis, and operational feedback as a connected decision-support workflow. Table 1 summarizes these layers, while Figure 2 illustrates how users, software services, data sources, and models interact.

At the application and presentation layer, the Flutter mobile client provides the operational entry point for growers. It handles field creation, crop-season inspection, weather and warning display, image upload, plant-protection and irrigation tasks, field notes, disease reporting, and forum communication. This layer is important because the platform is designed to support field decisions rather than only model execution.

At the data ingestion and storage layer, the Django REST backend receives mobile inputs and coordinates persistence in PostgreSQL. Field boundaries, crop seasons, weather records, phenology outputs, disease-risk outputs, fungicide and pesticide master data, tasks, notes, notices, messages, and forum records are stored as linked business entities. The crop season is the main data anchor because it connects field geometry, cultivar information, weather, phenology, disease risk, operations, and observations.

At the phenology and disease analytics layer, scheduled backend workflows retrieve weather data, transform hourly and daily weather variables, simulate grapevine phenology, evaluate spraying suitability, and construct disease-model requests. The independent FastAPI disease-model service receives weather, growth stage, cultivar susceptibility, disease targets, and fungicide-history inputs, then returns infection risks, stress risks, field risks, protection times, action recommendations, and treatment windows.

At the image-based diagnosis layer, uploaded field images are processed by backend image-analysis modules. YOLO, ViT or EfficientViT, UNet, and segmentation-related components support disease detection, recognition, segmentation, processed-image generation, and severity-related outputs. This layer complements weather-driven disease-risk analytics by adding symptom-

Table 1: Logical architecture layers of GrapeMaster.

Layer	Main components	Main responsibilities
Application and presentation layer	Dart/Flutter modules for fields, tasks, recognition, forum, maps, notes, authentication, weather, and notifications; Kotlin/Gradle and Java-compatible Android shell	Mobile interaction, field input, weather and warning display, task operation, image upload, and grower communication
Data ingestion and storage layer	Python, Django REST Framework, PostgreSQL, Django Channels, Django APScheduler, SimpleJWT, master-data modules	API routing, authentication, data persistence, scheduled data updates, WebSocket routing, and business records
Phenology and disease analytics layer	Python weather processing, Pandas, NumPy, GDD and Wang–Engel phenology model, spraying-suitability workflow, FastAPI disease-risk model	Weather transformation, growth-stage simulation, disease-risk simulation, treatment-window generation, and risk interpretation
Image-based diagnosis layer	Python, PyTorch, TorchVision, OpenCV, Pillow, Ultralytics YOLO/SAM, EfficientViT, UNet, segmentation, recognition and reporting modules	Disease-image upload, detection, recognition, segmentation, processed-image generation, and symptom-level evidence extraction
Task recommendation and alert layer	Notice services, task services, fungicide-history conversion, action recommendations, treatment windows	Warning delivery, task recommendation, plant-protection feedback, overdue handling, and recalculation after completed operations

level evidence from field observations.
At the task recommendation and alert layer, analytical outputs are converted into user-facing decisions. Warnings, treatment windows, recommendations, spraying-suitability indicators, and overdue task states are delivered through notices and task interfaces. Completed plant-protection tasks are converted into fungicide-history records, which are then used in later disease-risk recalculation. This feedback mechanism closes the management loop from model output to field operation and back to model input.
Figure 2 describes the interaction among users, data sources, software layers, frontend interfaces, backend services, and analytical models.

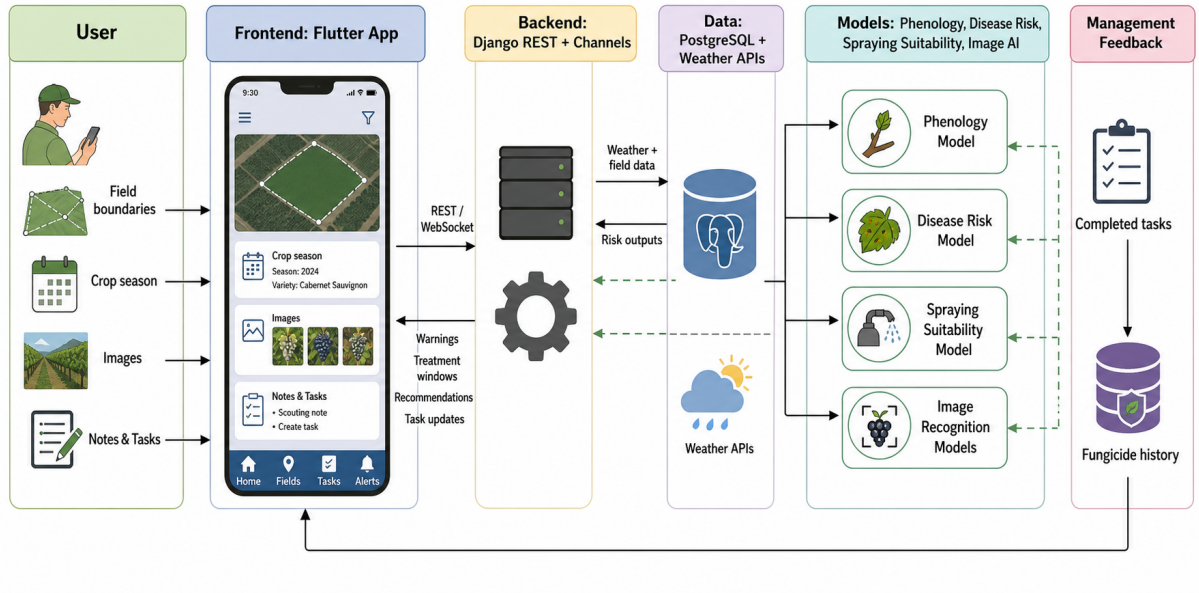

Figure 2: Interaction flow among users, mobile frontend, backend services, data sources, analytical models, and management feedback in GrapeMaster.

3 Implementation

3.1 Application and Presentation Layer Implementation

The application layer is implemented as a Flutter mobile client written in Dart. The Android project shell uses Kotlin and Gradle on the Java-compatible Android toolchain for platform integration, while the cross-platform user interface and business interaction logic remain in Flutter. Following the layered reporting style of end-to-end agricultural platforms, the client was designed as the operational entry point rather than only a visualization front end. The main navigation contains four modules: field management, task management, image-based recognition, and forum communication. The field module is the core workspace. It supports vineyard field creation and inspection, crop-season selection, weather display, growth-stage information, disease-risk review, notifications, field sharing, and recycle-bin operations. The task module supports plant-protection and irrigation workflows, including task creation, update, completion, deletion, and overdue-state handling. The recognition module supports image selection from camera or album, local image resizing and compression, image upload, disease recognition, severity inspection, disease reporting, historical diagnosis messages, and WebSocket-based intelligent-recognition interactions. The forum module supports grower communication through posts, comments, replies, and reply notifications.
The mobile client uses dio, http, and requests packages for REST requests, and web socket

channel support for real-time interactions. Local state such as user credentials, selected fields, selected crop seasons, and user preferences is stored through shared preferences. Location and map functions are implemented with AMap packages, geolocator, geojson, and flutter map. Image-related workflows use image picker, the Dart image package, flutter image compression, and multipart upload. Weather, phenology, disease-risk, task, note, notification, and forum records are rendered through native Flutter pages, while charts and media-related displays use packages such as fl chart, carousel slider, flutter svg, webview flutter, flutter pdfview, and video player. In this implementation, the mobile interface converts analytical outputs into field-oriented interactions: a warning can lead to a plant-protection task, a completed task can update the disease-model fungicide history, and an image diagnosis can lead to disease reporting or consultation.

3.2 Data Ingestion and Storage Implementation

The backend is implemented in Python with Django 4.2 and Django REST Framework, with PostgreSQL as the main structured database. Database access uses `psycopg2` and the Django ORM, while configuration and environment variables are handled with `python-decouple`. The service exposes REST endpoints for users, fields, crop seasons, master data, weather, phenology, disease risk, tasks, notes, notices, messages, forums, model outputs, image processing, application updates, and feedback. SimpleJWT and PyJWT are used for token-based mobile authentication. Django Channels provides WebSocket routing for notice delivery and intelligent-recognition sessions, while Django APScheduler and APScheduler support scheduled weather and model workflows. API documentation and schema support are provided through DRF schema tools and `drf-yasg`. Cross-origin access is handled through `django-cors-headers`.

The data model is organized around two primary agricultural entities: the vineyard field and the crop season. The field record stores the user, field name, area, GeoJSON boundary, centroid longitude and latitude, administrative region, sharing state, deletion state, notice state, and a generated field-boundary preview. When a field is saved, the backend parses the GeoJSON boundary, derives a boundary image, estimates the field area, calculates the centroid, and requests geocoding information from an external service. Geospatial processing uses Python packages such as Shapely, Fiona, GeoPandas, Matplotlib, and Geopy, while external service calls use Requests or HTTPX. The crop-season record links the field with cultivar, cultivation method, vine age, field coordinates, field size, and cultivar metadata. This record is the main relational anchor for weather records, phenology outputs, disease-risk outputs, fungicide and irrigation tasks, notes, warnings, and user-facing summaries.

Master-data tables store grape varieties, cultivar susceptibility, fungicides, pesticides, BBCH growth stages, cultivation methods, disease types, and reminder rules. The fungicide table is particularly important because task records are later transformed into disease-model inputs. It stores target disease, preventive protection days, curative protection days, efficacy parameters, suitable growth-stage range, dose information, active ingredients, and safety information. This design allows the backend to convert user-completed plant-protection tasks into structured fungicide-history records rather than treating them only as calendar events.

The implementation stores both normalized relational records and JSON model payloads. Weather, phenology, disease-risk, and disease-request-body records can therefore be served to the mobile client while preserving the original model inputs and outputs needed for recalculation, auditing, and future validation. This hybrid pattern is useful for a decision-support platform because model workflows often require nested time-series data that do not fit naturally into a single relational table.

3.3 Phenology and Disease Analytics Implementation

3.3.1 Weather Processing

For each active crop season, scheduled Python backend jobs retrieve historical and forecast weather using the centroid of the vineyard field. The weather workflow uses HTTP clients such as Requests, HTTPX, `openmeteo_requests`, `requests_cache`, and `retry_requests` to request historical data from the beginning of the current year to the previous day and combine these observations with a 16-day forecast. Hourly variables include air temperature, relative humidity, dew point, precipitation, wind speed, and wind gusts. Daily variables include maximum, minimum, and mean temperature, precipitation, maximum wind speed, maximum gust speed, sunrise time, and sunset time. Missing values are filled by carrying forward the previous available value before the records are used by analytical modules.
The backend converts these data into three working schemas using Pandas, NumPy, Python Dateutil, Pytz, and standard JSON processing. First, daily mean temperature is transformed into phenology-model input. Second, hourly weather is transformed into the disease-model schema, including datetime, dew point, precipitation, relative humidity, air temperature, and wind speed. Third, short-term hourly records are used to evaluate spraying suitability. This preprocessing stage is important because the platform must coordinate different model requirements while keeping a single crop-season data anchor.

3.3.2 Phenology Simulation

The phenology workflow is implemented in Python and estimates grapevine growth-stage progression from daily mean temperature and crop-season metadata. The implementation uses Pandas and NumPy to calculate a Wang–Engel temperature response function and daily growing degree days (GDD). Daily GDD values are accumulated into `GDDCUSUM`. The accumulated heat is then mapped to BBCH principal stages using region-, cultivar-, or maturity-specific thresholds stored in the model parameter file. The output contains date, GDD, accumulated GDD, BBCH principal stage, and one-digit BBCH class. The backend joins these outputs with BBCH master data to add growth-stage codes and Chinese and English descriptions for mobile display.
The phenology output has two functions in the platform. It provides growers with a crop-development timeline, and it supplies the disease-risk model with a daily growth-stage sequence. This connection is essential because grapevine diseases are not only weather dependent; their relevance and risk interpretation also depend on whether the crop is within a susceptible phenological window.

3.3.3 Disease-Risk Simulation

Disease-risk simulation is provided by an independent Python FastAPI service with a `/simulate` endpoint. The service uses Uvicorn for ASGI serving and Pydantic models for request validation and typed schema definition. The Django backend constructs a crop-season-level request body containing crop-season identifiers, region, crop EPPO code, latitude, longitude, request date, hourly weather, growth-stage sequence, historical fungicide applications, target disease codes, cultivar susceptibility values, requested features, and date ranges. In the current vineyard workflow, the main target disease codes are `PLASVI` for grape downy mildew and `UNCINE` for grape powdery mildew. The model service also contains additional grape disease modules, but the operational backend workflow primarily consumes downy mildew and powdery mildew outputs.
Inside the disease-model service, the request body is parsed into a crop-season object. The service dynamically loads the crop implementation according to the crop EPPO code and dynamically initializes the requested disease modules. Numerical and tabular computation is performed

mainly with Pandas and NumPy, while date handling uses Python Dateutil and Pytz. The grape disease implementation calculates daily weather favorability from hourly weather. For infection favorability, the model uses temperature response, relative-humidity-derived wetness duration, and disease-specific thresholds. It then applies growth-stage correction, cultivar susceptibility correction, and fungicide-effect correction. Daily favorability values are classified into infection-risk categories, and a rolling short-term aggregation is used to derive more stable disease-risk states.

The disease-risk workflow also includes disease-onset logic. It estimates primary inoculum availability, inoculum maturation, spore release, and potential infection events using weather and disease parameters. Dates before modeled disease onset are downgraded to unfavorable states, and dates outside the configured BBCH disease window are treated as non-seasonal. Fungicide records can convert otherwise risky days into protected days when preventive or curative protection effects are active.

The model returns daily infection risks, stress risks, field risks, stress protection times, action recommendations, and treatment windows. Field risk is derived from the individual disease risks by selecting the highest relevant unprotected risk while preserving fully protected states when all target diseases are protected. The backend compresses these outputs into date-aligned arrays and stores them in the disease-data table for mobile display and later use.

3.3.4 Spraying Suitability Assessment

The spraying-suitability workflow evaluates whether short-term weather conditions support plant-protection operations. The implementation considers air temperature, current precipitation, wind speed, relative humidity, precipitation expected in the next hours, cultivation method, sunrise time, and sunset time. The resulting suitability indicators are calculated for the forecast horizon and stored with crop-weather records. These indicators allow the system to connect disease-risk warnings with practical execution conditions, reducing the chance that a recommended treatment is scheduled under unsuitable weather.

3.4 Image-Based Diagnosis Implementation

The image-based diagnosis layer is implemented in Python as a set of backend image-processing endpoints and model modules. The mobile client uploads images through multipart requests after local selection and preprocessing. For disease classification, the backend uses PyTorch and an EfficientViT-based classifier that maps uploaded grape images to disease or healthy categories and returns corresponding prevention or management text. Pillow is used for image loading in the classifier workflow, while model execution uses PyTorch tensors and EfficientViT model-zoo utilities. The current category mapping includes grape leaf blight, grape black rot, grape Esca or black measles, healthy grape, and grapevine yellowing.

The platform also includes a segmentation-based severity pipeline. The pipeline uses OpenCV, NumPy, PyTorch, TorchVision transforms, and Ultralytics model components. It first applies YOLO to locate leaf or leaf-disc regions, then uses MobileSAM to segment the leaf region from the detected box, and finally applies a UNet model to segment lesion and leaf pixels in the cropped region. Severity is calculated as the percentage of lesion pixels relative to the sum of lesion and leaf pixels:

$$\text{Severity} = \frac{N_{\text{lesion}}}{N_{\text{lesion}} + N_{\text{leaf}}} \times 100. \tag{1}$$

The output includes a processed image with bounding boxes and severity labels, together with per-region severity values. This image pathway complements weather-driven disease-risk analytics by adding symptom-level evidence from field observations. It also provides a route for users to report observed disease events, which can support future validation of model warnings.

3.5 Task Recommendation and Alert Implementation

Task recommendation and alert functions connect analytical outputs with field operations. Disease-risk outputs, phenology changes, treatment windows, spraying-suitability indicators, field-sharing events, and task states are stored by the backend and exposed to the mobile application through warning, notification, and task interfaces. The notice service stores title, message text, importance, read state, user, and crop season. WebSocket channels are used to deliver timely notice updates to the mobile client. Users can mark notices as read, accept shared fields, inspect crop-season warnings, and create plant-protection or irrigation tasks from warning contexts.

Plant-protection and irrigation tasks are implemented as persistent business objects. Each task stores crop season, execution date, task type, task status, field name, and task-specific parameters. If a task remains unexecuted after its execution date, the backend marks it as overdue. For plant-protection tasks, linked fungicide records store product name, dose, dose unit, and total unit. When a user creates, updates, deletes, or completes a plant-protection task, the backend compares the fungicide list and execution date with the previous record and updates the disease-model fungicide history when needed.

Completed plant-protection tasks are converted into applied-fungicide inputs for subsequent disease-risk recalculation. The backend derives application date, target disease, preventive protection period, curative protection period, and efficacy-related parameters from the task record and fungicide master data. This produces a feedback loop in which a field operation changes the future disease-risk trajectory. In other words, GrapeMaster does not only recommend actions; it also records whether actions were completed and incorporates those actions into later risk simulation.

4 Deployment

Deployment statistics, operating regions, vineyard numbers, crop-season counts, user numbers, disease-simulation counts, image uploads, task records, and notification counts should be added from operational logs before submission.

5 Features and Functionalities

This section describes the major user-facing functions implemented in GrapeMaster and the way these functions are presented in the mobile application. The section focuses on functional behavior rather than internal algorithms. In the current implementation, the mobile client is organized around four principal workspaces: vineyard field management, task management, intelligent recognition, and grower communication. These workspaces are connected by a crop-season context, so that weather information, phenological status, disease warning, image evidence, field notes, and completed operations are displayed and updated for the same vineyard season.

5.1 Vineyard Field and Crop-Season Management

The field-management function provides the spatial entry point of the platform. Users can create, inspect, edit, share, delete, recover, and select vineyard fields. During field creation, the mobile client supports map-assisted input and records the field name, boundary geometry, area, centroid longitude and latitude, administrative region, and a generated boundary preview. After a field has been created, it is displayed as a selectable field card in the mobile workspace. Entering a field opens the field-specific management page, where users can inspect crop seasons, weather, warnings, notes, tasks, and related field settings.

The platform treats field sharing as a managed collaboration function rather than only a data export operation. A user can invite another account to access a field. The recipient receives

a field-sharing notice, accepts the invitation from the notification interface, and then sees the shared field in the operational field list. Deleted fields are moved into a recycle-bin workflow so that accidental deletion does not immediately remove the field from user interaction. This supports practical field management in situations where vineyards are operated by multiple users or where historical field information must remain recoverable.

The crop-season function links a vineyard field with grape variety, cultivation method, vine age, field area, and coordinates. It is the central working unit for the platform. Weather retrieval, phenology simulation, disease-risk calculation, plant-protection tasks, irrigation tasks, notes, and alerts are all associated with a crop season. This design is important for perennial crop management because the same vineyard field may be used across multiple production years, while each season must maintain separate weather records, growth-stage trajectories, disease-risk outputs, and field-operation histories.

Within the crop-season workspace, users can view the selected cultivar and cultivation method, inspect the current growth-stage context, open disease-warning information, review weather details, and enter task or note pages without manually reselecting the field. The practical function of the workspace is to make the crop season the user's decision context. Instead of presenting weather, disease, notes, and tasks as isolated pages, the system keeps them attached to the same field-season object.

5.2 Weather, Phenology, and Spraying-Suitability Display

The weather function provides field- and crop-season-specific environmental information. The backend retrieves and stores hourly and daily weather variables according to the field centroid, while the mobile client presents these data as near-term weather summaries, hourly weather records, and multi-day forecast views. Hourly displays include temperature, relative humidity, precipitation, and wind speed. Daily displays include precipitation, maximum and minimum temperature, mean temperature, maximum wind speed, gust-related indicators, and weather-status information. These views allow users to inspect the local environmental conditions that explain disease warnings and task feasibility.

The phenology function displays predicted grapevine growth stages for the selected crop season. The backend calculates daily GDD, accumulated GDD, BBCH principal stages, one-digit BBCH stages, and growth-stage descriptions, and the mobile client presents the resulting stage information to users. The displayed growth-stage information has two direct management roles. First, it gives users a seasonal development reference when inspecting the field. Second, it explains why disease warnings or plant-protection recommendations may become more or less relevant as the crop enters susceptible growth stages.

The spraying-suitability function converts short-term weather conditions into practical field-operation guidance. It considers temperature, precipitation, wind speed, relative humidity, near-future rainfall, cultivation method, sunrise, and sunset. The mobile client can therefore display not only whether disease pressure is present, but also whether the near-term field conditions are suitable for spraying. This function is especially important when a treatment window is open but rainfall, wind, or unsuitable timing may reduce spray effectiveness or prevent safe execution.

5.3 Disease-Risk Warning and Treatment-Window Review

The disease-warning function presents model-derived disease-risk information for the selected crop season. The backend stores date-aligned disease-risk outputs for target grape diseases, including downy mildew and powdery mildew in the main operational workflow. The mobile client retrieves and displays disease-risk summaries, infection-risk codes, stress-risk codes, field-risk codes, action recommendations, and treatment windows. The purpose of this function is to make daily model outputs readable as management information rather than as raw numerical arrays.

The warning display separates several kinds of information that growers need to interpret together. Infection risk describes whether weather and disease conditions are favorable for infection. Stress risk summarizes disease-related pressure under recent or short-term conditions. Field risk integrates the disease-specific results into a crop-season-level warning state. Protection-related states indicate whether previous fungicide applications are still considered active by the model. Action recommendations translate these states into practical guidance, including whether treatment should be considered, delayed, or regarded as already protected.

The treatment-window function translates model risk into management timing. A future window indicates that a recommendation is approaching, a current window indicates that the recommended treatment period is active, and a missed window indicates that the suggested timing has passed while the risk remains relevant. This design reduces the burden on users to infer timing from weather, phenology, and risk codes. It also provides a direct bridge from disease warning to task creation, because a user can use the warning context to schedule a plant-protection operation.

The warning function is connected with notifications. When phenology changes or disease-risk states trigger an alert, the backend generates user-specific notices. These notices can be reviewed, marked as read, or used as entry points for related operations such as task creation. In this way, disease-risk information is exposed through both the field workspace and the notification workflow, which supports users who do not continuously inspect each field page.

5.4 Plant-Protection and Irrigation Task Management

The task module supports two major operation types: plant protection and irrigation. Users can create, inspect, edit, complete, and delete tasks. Each task is associated with a crop season and contains execution time, field name, task status, and operation-specific parameters. The task list allows users to inspect planned, completed, and overdue operations for the selected crop season or user account. Tasks that remain unexecuted after their scheduled execution date can be marked as overdue by the backend, allowing the interface to distinguish missed operations from pending ones.

Plant-protection tasks include container type, sprayer or mixer capacity, water consumption, execution date, task status, and one or more fungicide or pesticide records. The mobile task-creation workflow supports both large-container and sprayer-based mixing modes. Fungicide and pesticide records include product name, dose, dose unit, and total unit, allowing the task to represent the practical spray mixture rather than only a generic operation name. Users can revise task parameters before execution and mark tasks as completed after the field operation is carried out.

Once a plant-protection task is completed, the backend converts the operation into applied-fungicide history for the disease model. The completed task supplies the application date, target disease, and protection parameters derived from fungicide master data. This is a key functional difference between GrapeMaster and a simple task calendar: completed operations are not only recorded for user memory, but also change the disease-risk model state by adding protection effects. Subsequent disease-risk recalculation can therefore reflect whether users have actually acted on previous recommendations.

Irrigation tasks include irrigation method, growth-stage period, irrigation duration, irrigation dosage, execution time, and task status. This allows users to manage water-related operations in the same operational environment as plant protection, keeping crop-season activities organized around the same field and crop context.

5.5 Image-Based Disease Assistance and Severity Estimation

The recognition module supports field-image-based assistance. Users can select an image from the camera or gallery, preview it in the mobile client, and upload it for recognition. Before

upload, the image can be resized and compressed to reduce transmission load. The backend image-recognition service classifies grape images into disease or healthy categories and returns corresponding diagnostic or management text to the user. The current recognition output includes grape leaf blight, grape black rot, grape Esca or black measles, healthy grape, and grapevine yellowing categories.

The severity-estimation function provides lesion-level analysis for disease images. The backend applies a detection-segmentation pipeline to identify leaf regions, segment the leaf and lesion areas, generate a processed image, and calculate a severity percentage for each detected region. The mobile interface can display the returned processed image and severity information so that users can inspect both the recognition result and the visual evidence used to support it. This output helps users move from qualitative symptom observation to a quantitative estimate of disease severity.

The recognition workflow is designed as a complement to model-based disease warning. Weather-driven models provide risk forecasts even before visible symptoms are present, while field images provide symptom-level evidence once disease signs are observed. The platform keeps these functions in the same mobile environment, allowing users to compare warning information, field symptoms, and their own notes without switching to a separate diagnostic tool.

5.6 Field Notes, Growth-Stage Notes, and Disease Records

The note module supports user-entered field records. Users can create ordinary crop notes, growth-stage notes, and disease-incidence notes. These records can include text, date, crop-season context, disease or growth-stage selections, and images. Notes provide a lightweight field diary for management events and observations that are not necessarily formal tasks, such as canopy observations, visible symptoms, abnormal weather effects, or informal inspection results. Growth-stage notes allow users to record observed crop development and compare field observations with model-predicted growth stages. Disease notes allow users to record disease type, incidence-related information, and supporting images. These records are useful for later interpretation of model results and can support future validation by linking model-predicted risk periods with user-observed field symptoms. From the user's perspective, the note function provides a persistent memory of field observations; from the system perspective, it creates structured field evidence that can be connected with disease warnings and image-recognition records.

5.7 Disease Reporting and Warning Feedback

The reporting function allows users to submit observed disease events from the field. Reports can include user identity, field context, disease information, geographic information, and images. The backend can convert such reports into warning records and notify relevant users. This function gives the system an observation channel in addition to weather-driven simulation and image classification.

From a platform perspective, disease reporting is important because it supports two-way information flow. The system sends warnings to users, but users can also send field evidence back to the system. This provides a basis for later model validation, regional warning refinement, and expert review. It also allows the platform to distinguish between purely simulated risk and user-confirmed disease occurrence, which is useful when evaluating whether warnings correspond to field observations.

5.8 Notifications, Field Sharing, and User Interaction

The notification function provides user-specific alerts for disease warnings, task-related reminders, phenology-related operations, and field-sharing events. Notifications contain title, text, creation time, read state, importance, user, and crop-season association. The mobile client supports notification review, single-notice reading, grouped reading, important-notice review, and

all-read operations. Some notifications provide direct actions, such as accepting a shared field or creating a task from a warning.
Real-time notification delivery is supported through WebSocket channels. This reduces the delay between backend event generation and user awareness, which is important for time-sensitive plant-protection recommendations. Notification settings can also be stored locally in the mobile client so that users can control how reminders are presented.
The platform also includes grower communication functions through the forum module. Users can create posts, upload images, inspect post details, add comments, reply to comments, and review reply notifications. This function supports practical information exchange among users, especially for field observations, disease symptoms, operation experience, and management questions that may not be fully captured by model outputs.

5.9 User Account, Localization, and Auxiliary Services

The user-account function supports login, profile information, local credential storage, and user-specific access to fields, tasks, notices, messages, and reports. Authentication allows the backend to associate operational records with specific users and to control access to field information. Local storage in the mobile client keeps selected user and field context available across sessions, reducing repeated input during field use.
The mobile application includes Chinese and English localization resources, allowing interface text to be presented in different languages. Auxiliary services include suggestion and feedback submission, historical message review, PDF or document access, and application update support. These functions are not the core analytical contribution of the platform, but they improve the completeness of the mobile decision-support environment by supporting maintenance, communication, documentation, and user feedback.

6 Discussion

The literature on vineyard disease management and agricultural decision support has repeatedly shown that useful decision support depends not only on predictive algorithms, but also on whether monitoring, model outputs, management timing, and user interaction are connected in a form that growers can act on [McCown, 2002, Rose et al., 2016, Rossi et al., 2014, Walling and Vaneeckhaute, 2020]. Recent work on farm management information systems, digital agriculture architectures, and viticulture decision support has further emphasized that system value emerges from the integration of data management, analytics, interfaces, and business processes rather than from isolated software functions alone [Fountas et al., 2015, Tummers et al., 2019, 2021, Uyar et al., 2024, Knowling et al., 2021]. From this perspective, GrapeMaster is better interpreted as an integrated mobile software environment for crop-season management than as a standalone disease-warning model. The platform combines weather retrieval, phenology simulation, disease-risk modeling, image-based disease assistance, task execution, and notification delivery within one operational workflow. This system-level integration addresses a long-recognized fragmentation problem in agricultural DSS deployment, where analytical functions, records, and management actions are often distributed across separate tools and therefore lose decision value at the point of use [McCown, 2002, Fountas et al., 2015, Tummers et al., 2021, Walling and Vaneeckhaute, 2020]. In practical terms, the architecture moves the platform from model reporting toward decision support because warnings, treatment windows, field observations, and field operations are represented within the same crop-season context.

6.1 System Integration

An important feature of the platform is that it organizes analytical and operational data around the crop season rather than around a single model run or an isolated field record. This de-

sign matches the biological and managerial structure of vineyard disease control, in which risk depends on field location, cultivar susceptibility, phenological stage, recent weather, and prior treatment history [Peng et al., 2024, Kunova et al., 2021, Puelles et al., 2024]. By making the crop season the central unit, GrapeMaster connects field geometry, weather updates, BBCH-linked phenology outputs, disease warnings, fungicide applications, irrigation tasks, notes, and notices without requiring users to reconstruct context across separate modules. In software terms, this is consistent with the reference-architecture view that FMISs should integrate data management, operational records, and decision support within one coherent application structure [Tummers et al., 2021], and with viticulture modeling work that frames useful decision support as an exercise in linking data streams to seasonal decisions rather than storing them in parallel silos [Knowling et al., 2021].

The crop-season-centered design also gives the platform a temporal logic that is often absent from generic recording systems. Phenology is not merely displayed but used to interpret disease relevance. Fungicide applications are not merely archived but converted into model inputs that affect later risk states. Short-term weather is not merely retrieved for inspection but transformed into spraying-suitability judgments and treatment-timing cues. In this sense, the platform follows a principle emphasized in vineyard DSS research and integrated pest-management typologies: disease support is most useful when weather, crop development, and intervention timing are interpreted together rather than separately [Rosa et al., 1993, Rossi et al., 2014, Marinko et al., 2024, Helps et al., 2024, Knowling et al., 2021].

This integrated organization is particularly relevant for vineyards because the same field persists across years, whereas the disease-relevant state of the crop changes continuously within a season. A field-only or disease-only data model is therefore too narrow. The crop season provides an intermediate unit that is stable enough for operational management while remaining specific to annual weather, phenology, and protection history. This design choice is one of the main reasons the platform can support traceability from field creation to warning generation, task execution, and later management review.

6.2 Operational Value

The operational value of the platform lies in its combination of forecast-oriented and observation-oriented evidence. Weather-driven disease models have long been used to support fungicide timing, whereas recent computer-vision studies and sensing reviews have shown that image classification, detection, lesion segmentation, and proximal sensing can support disease recognition and severity estimation in grape production [Puelles et al., 2024, Kunduracioglu and Pacal, 2024, Shi et al., 2023, Liu et al., 2024, Portela et al., 2024]. GrapeMaster brings these two directions into the same mobile environment. Model outputs allow the system to warn before visible symptoms become severe, whereas uploaded field images, disease notes, and disease reports provide symptom-level evidence once disease can be directly observed. This combination is valuable because visual symptoms alone do not provide treatment timing or protection-state interpretation, while weather-driven risk values alone do not document whether symptoms are already present or how severe they appear in the field [Rossi et al., 2014, Shi et al., 2023, Puelles et al., 2024, Portela et al., 2024].

The platform also adds value by translating analytics into management actions. Many agricultural DSSs produce outputs that are scientifically meaningful but operationally indirect, such as risk indices, infection events, or model states that still require interpretation before they can guide field activity [Helps et al., 2024, Walling and Vaneeckhaute, 2020]. In GrapeMaster, this translation is implemented through treatment windows, recommendations, spray-suitability indicators, and task objects. The task object is especially important because it gives the warning system a concrete operational endpoint: a predicted risk can become a scheduled operation, and a completed operation can become revised disease protection status. This is a stronger management link than simply displaying a chart or forecast.

Another practical feature is that the platform does not restrict decision support to plant protection alone. The same crop-season workspace can include irrigation tasks, which is relevant in vineyard management because disease, canopy condition, water status, and seasonal crop development interact in practice even when they are modelled separately [Kang et al., 2023, Ortega-Farias and Riveros-Burgos, 2019, Knowling et al., 2021]. Although the present manuscript focuses on disease warning, the inclusion of irrigation functions suggests that the platform can evolve into a broader viticultural management environment rather than remaining a single-purpose warning application. This wider operational scope is aligned with current views of digital agriculture systems as environments for coordinating data, decisions, and actions across multiple farm processes [Uyar et al., 2024, Tummers et al., 2021].

6.3 Validation Needs

Several validation tasks remain necessary before the platform can be presented as a fully validated operational decision-support system. First, the disease-warning workflow requires field-scale evaluation across sites, seasons, and cultivars to assess whether simulated risk states, treatment windows, and fungicide-protection logic correspond to observed disease development and management outcomes [Puelles et al., 2024, Mian et al., 2021, Helps et al., 2024]. In particular, it would be useful to evaluate not only whether infection periods are predicted correctly, but also whether the use of the platform changes the number, timing, or effectiveness of interventions relative to existing grower practice, because recent DSS evaluation work has argued that practical value should be measured in terms of economic and environmental consequences as well as predictive behavior [Helps et al., 2024]. This kind of field-oriented validation is especially important in viticulture, where earlier DSS studies have shown that operational tools gain credibility when they are tested against actual spray practice and field performance rather than against model behavior alone [Gil et al., 2011, Knowling et al., 2021].
Second, the image-recognition and severity-estimation modules require explicit quantitative benchmarking under realistic vineyard imaging conditions, because reported performance in controlled datasets does not automatically transfer to operational field use [Kunduracioglu and Pacal, 2024, Shi et al., 2023, Portela et al., 2024]. Such validation should ideally include disease class imbalance, mixed symptom backgrounds, different mobile-device cameras, and repeated observations across the season. Third, the platform would benefit from deployment-scale evidence on service latency, update robustness, notification timeliness, and task-completion behavior, because the practical success of agricultural DSS platforms depends on reliability, maintainability, and clarity of interaction as much as on analytical sophistication [Tummers et al., 2021, Walling and Vaneeckhaute, 2020, Ara et al., 2021, Zhai et al., 2020].
Finally, user-centered evaluation is needed to determine whether the integrated workflow actually improves timing decisions, reduces unnecessary interventions, or changes how growers interpret disease risk during the season. This is not a minor auxiliary issue. Research on agricultural DSS adoption, interface design, and algorithm aversion suggests that even well-designed analytical systems may be underused when users perceive outputs as opaque, poorly timed, weakly integrated into practice, or less trustworthy than human advice [Rose et al., 2018, Grant et al., 2026, Gent et al., 2011, Gutiérrez et al., 2019]. For GrapeMaster, this implies that future evaluation should include not only model metrics but also workflow metrics such as notice response, task conversion, completion delay, repeated seasonal use, perceived usefulness, and clarity of presentation. These are the steps needed to move the manuscript from a technically complete platform description toward a rigorously validated systems paper.

6.4 Limitations

Several limitations should be stated explicitly. The present study documents an implemented system architecture, but it does not yet provide field-scale evidence that the integrated work-

flow consistently improves agronomic outcomes across regions, cultivars, and seasons; in this sense, the platform demonstrates technical integration and operational design rather than full agronomic effectiveness, a distinction that remains central in DSS research because technical completeness alone does not ensure practical success when calibration, trust, usability, and fit with real decision contexts are still uncertain [McCown, 2002, Walling and Vaneeckhaute, 2020, Rose et al., 2018, Gent et al., 2011, Knowling et al., 2021]. The embedded analytical modules also remain subject to the transferability limits reported in the underlying literature, since vineyard disease forecasts are sensitive to climatic region, disease pressure, calibration assumptions, and the relationship between simulated infection periods and observed symptoms, while phenology models remain influenced by cultivar and environmental context [Puelles et al., 2024, Mian et al., 2021, Caffarra et al., 2014, Molitor et al., 2020]. The same caution applies to the image-based functions: recent studies show strong progress in grape disease recognition and vineyard sensing, but many results still depend on specific datasets, sensors, and imaging conditions, so operational use requires evaluation of robustness to lighting, symptom similarity, device heterogeneity, background complexity, user capture behavior, and the presentation of risk and recommendations [Kunduracioglu and Pacal, 2024, Portela et al., 2024, Gutiérrez et al., 2019]. Finally, the manuscript does not yet report deployment-scale evidence such as long-term user retention, repeated seasonal use, service resilience, or comparative gains relative to existing advisory practice, and survey work in digital agriculture continues to show that interoperability, scalability, accessibility, usability, and service integration remain persistent barriers [Zhai et al., 2020, Tummers et al., 2021]. The paper is therefore best read as a system paper that defines and implements an integrated vineyard decision-support architecture rather than as a final validation study of production-scale performance.

References

Iffat Ara, L. R. Turner, Matthew T. Harrison, Marta Monjardino, Philip DeVoil, and Daniel Rodríguez. Application, adoption and opportunities for improving decision support systems in irrigated agriculture: A review. *Agricultural Water Management*, 257:107161, 2021. doi: 10.1016/j.agwat.2021.107161.

Mehmet E. Bakir, Andres R. Perea, Micah Funk, Sajidur Rahman, Maximiliano J. Spetter, Lara Macon, Andrew Cox, Richard E. Estell, Huiping Cao, Andres F. Cibils, Sheri A. Spiegal, Brandon T. Bestelmeyer, and Santiago A. Utsumi. A scalable, end-to-end iot and remote sensing platform for precision rangeland and livestock management. *Computers and Electronics in Agriculture*, 247:111615, 2026. doi: 10.1016/j.compag.2026.111615.

Amelia Caffarra, Massimo Rinaldi, Emanuele Eccel, Vittorio Rossi, and Ilaria Pertot. A comparison of different modelling solutions for studying grapevine phenology under present and future climate scenarios. *Agricultural and Forest Meteorology*, 195–196:192–205, 2014. doi: 10.1016/j.agrformet.2014.05.011.

B. G. Coombe. Growth stages of the grapevine: Adoption of a system for identifying grapevine growth stages. *Australian Journal of Grape and Wine Research*, 1(2):104–110, 1995. doi: 10.1111/j.1755-0238.1995.tb00086.x.

Spyros Fountas, Giacomo Carli, Claus Aage Grøn Sørensen, Zisis Tsiropoulos, Christos Cavalaris, Anna Vatsanidou, Basil Liakos, Maurizio Canavari, Jens Wiebensohn, and Bruno Tisserye. Farm management information systems: Current situation and future perspectives. *Computers and Electronics in Agriculture*, 115:40–50, 2015. doi: 10.1016/j.compag.2015.05.011.

David H. Gent, Erick De Wolf, and Sarah J. Pethybridge. Perceptions of risk, risk aversion, and barriers to adoption of decision support systems and integrated pest management: an introduction. *Phytopathology*, 101(6):640–643, 2011. doi: 10.1094/PHYTO-04-10-0124.

Emilio Gil, Jordi Llorens, Andrew J. Landers, Jordi Llop, and Llorenç Giralt. Field validation of dosaviña, a decision support system to determine the optimal volume rate for pesticide application in vineyards. *European Journal of Agronomy*, 35(1):33–46, 2011. doi: 10.1016/j.eja.2011.03.005.

Jack H. Grant, Dorothee Scharpenberg, and Louise Manning. Algorithm aversion in agricultural decision-making: Trust dynamics, barriers, and fertiliser-related decision support. *Agricultural Systems*, 233:104630, 2026. doi: 10.1016/j.agsy.2025.104630.

Francisco Gutiérrez, Nyi Nyi Htun, Florian Schlenz, Aikaterini Kasimati, and Katrien Verbert. A review of visualisations in agricultural decision support systems: An HCI perspective. *Computers and Electronics in Agriculture*, 163:104844, 2019. doi: 10.1016/j.compag.2019.05.053.

Joseph C. Helps, Frank van den Bosch, Neil Paveley, Lise Nistrup Jørgensen, Niels Holst, and Alice E. Milne. A framework for evaluating the value of agricultural pest management decision support systems. *European Journal of Plant Pathology*, 169:887–902, 2024. doi: 10.1007/s10658-024-02878-1.

Chenchen Kang, Geraldine Diverres, Manoj Karkee, Qin Zhang, and Markus Keller. Decision-support system for precision regulated deficit irrigation management for wine grapes. *Computers and Electronics in Agriculture*, 208:107777, 2023. doi: 10.1016/j.compag.2023.107777.

Matthew J. Knowling, Bree Bennett, Bertram Ostendorf, Seth Westra, Rob R. Walker, Anne Pellegrino, Everard J. Edwards, Cassandra Collins, Vinay Pagay, and Dylan Grigg. Bridging the gap between data and decisions: A review of process-based models for viticulture. *Agricultural Systems*, 193:103209, 2021. doi: 10.1016/j.agsy.2021.103209.

Ismail Kunduracioglu and Ishak Pacal. Advancements in deep learning for accurate classification of grape leaves and diagnosis of grape diseases. *Journal of Plant Diseases and Protection*, 131:1061–1080, 2024. doi: 10.1007/s41348-024-00896-z.

Andrea Kunova, Cristina Pizzatti, Marco Saracchi, Matias Pasquali, and Paolo Cortesi. Grapevine powdery mildew: Fungicides for its management and advances in molecular detection of markers associated with resistance. *Microorganisms*, 9(7):1541, 2021. doi: 10.3390/microorganisms9071541.

Bin Liu, Zhaoyang Ding, Li Tian, Dongjian He, Shuqin Li, and Hongyan Wang. Grape leaf disease identification using improved deep convolutional neural networks. *Frontiers in Plant Science*, 11:1082, 2020. doi: 10.3389/fpls.2020.01082.

Yifan Liu, Qiudong Yu, and Shuze Geng. Real-time and lightweight detection of grape diseases based on fusion transformer YOLO. *Frontiers in Plant Science*, 15:1269423, 2024. doi: 10.3389/fpls.2024.1269423.

D. H. Lorenz, K. W. Eichhorn, H. Bleiholder, R. Klose, U. Meier, and E. Weber. Growth stages of the grapevine: Phenological growth stages of the grapevine (Vitis vinifera l. ssp. vinifera)–codes and descriptions according to the extended BBCH scale. *Australian Journal of Grape and Wine Research*, 1(2):100–103, 1995. doi: 10.1111/j.1755-0238.1995.tb00085.x.

Jurij Marinko, Bojan Blažica, Lise Nistrup Jørgensen, Niels Matzen, Mark Ramsden, and Marko Debeljak. Typology for decision support systems in integrated pest management and its implementation as a web application. *Agronomy*, 14(3):485, 2024. doi: 10.3390/agronomy14030485.

R. L. McCown. Locating agricultural decision support systems in the troubled past and socio-technical complexity of 'models for management'. *Agricultural Systems*, 74(1):11–25, 2002. doi: 10.1016/S0308-521X(02)00020-3.

Giovanni Mian, Edoardo Buso, and Matteo Tonon. Decision support systems for downy mildew (Plasmopara viticola) control in grapevine: Short comparison review. *Asian Research Journal of Agriculture*, 14(2):12–20, 2021. doi: 10.9734/arja/2021/v14i230120.

Daniel Molitor, Jürgen Junk, Danièle Evers, Lucien Hoffmann, and Marco Beyer. A high-resolution cumulative degree day-based model to simulate phenological development of grapevine. *American Journal of Enology and Viticulture*, 65(1):72–80, 2014. doi: 10.5344/ajev.2013.13066.

Daniel Molitor, Amelia Caffarra, Peter Sinigoj, Ilaria Pertot, Lucien Hoffmann, and Jürgen Junk. UniPhen: a unified high resolution model approach to simulate the phenological development of a broad range of grape cultivars as well as a potential new bioclimatic indicator. *Agricultural and Forest Meteorology*, 291:108024, 2020. doi: 10.1016/j.agrformet.2020.108024.

Simone Orlandini, Bruno Gozzini, Matteo Rosa, Elisabeth Egger, Paolo Storchi, and Giampiero Maracchi. A computer program to improve the control of grapevine downy mildew. *Computers and Electronics in Agriculture*, 12(4):311–322, 1995. doi: 10.1016/0168-1699(95)00007-Q.

Samuel Ortega-Farias and Camilo Riveros-Burgos. Modeling phenology of four grapevine cultivars (Vitis vinifera l.) in mediterranean climate conditions. *Scientia Horticulturae*, 250:38–44, 2019. doi: 10.1016/j.scienta.2019.02.025.

Junbo Peng, Xuncheng Wang, and Hui Wang. Advances in understanding grapevine downy mildew: From pathogen infection to disease management. *Molecular Plant Pathology*, 25(1): e13401, 2024. doi: 10.1111/mpp.13401.

Fernando Portela, Joaquim J. Sousa, Cláudio Araújo-Paredes, Emanuel Peres, Raul Morais, and Luís Pádua. A systematic review on the advancements in remote sensing and proximity tools for grapevine disease detection. *Sensors*, 24(24):8172, 2024. doi: 10.3390/s24248172.

Miguel Puelles, Julia Arbizu-Milagro, Francisco J. Castillo-Ruiz, and J. M. Peña. Predictive models for grape downy mildew (Plasmopara viticola) as a decision support system in mediterranean conditions. *Crop Protection*, 175:106450, 2024. doi: 10.1016/j.cropro.2023.106450.

Mati Ur Rahman, Xia Liu, Xiping Wang, and Ben Fan. Grapevine gray mold disease: infection, defense and management. *Horticulture Research*, 11(9):uhae182, 2024. doi: 10.1093/hr/uhae182.

Matteo Rosa, Roberto Genesio, Bruno Gozzini, Giampiero Maracchi, and Simone Orlandini. PLASMO: a computer program for grapevine downy mildew development forecasting. *Computers and Electronics in Agriculture*, 9(3):205–215, 1993. doi: 10.1016/0168-1699(93)90039-4.

David C. Rose, William J. Sutherland, Caroline Parker, Matt Lobley, Michael Winter, Carol Morris, Susanne Twining, Claire Ffoulkes, Tatsuya Amano, and Lynn V. Dicks. Decision support tools for agriculture: Towards effective design and delivery. *Agricultural Systems*, 149:165–174, 2016. doi: 10.1016/j.agsy.2016.09.009.

David C. Rose, Caroline Parker, Joe Fodey, Caroline Park, William Sutherland, and Lynn V. Dicks. Involving stakeholders in agricultural decision support systems: Improving user-centred design. *International Journal of Agricultural Management*, 6(3-4):80–89, 2018. doi: 10.5836/ijam/2017-06-80.

Vittorio Rossi, Fabio Salinari, Stefano Poni, Tito Caffi, and Teresa Bettati. Addressing the implementation problem in agricultural decision support systems: the example of vite.net. *Computers and Electronics in Agriculture*, 100:88–99, 2014. doi: 10.1016/j.compag.2013.10.011.

Serge Savary, Laetitia Willocquet, Sarah Jane Pethybridge, Paul Esker, Neil McRoberts, and Andy Nelson. The global burden of pathogens and pests on major food crops. *Nature Ecology & Evolution*, 3:430–439, 2019. doi: 10.1038/s41559-018-0793-y.

Tingting Shi, Yongmin Liu, Xinying Zheng, Kui Hu, Hao Huang, Hanlin Liu, and Hongxu Huang. Recent advances in plant disease severity assessment using convolutional neural networks. *Scientific Reports*, 13:2336, 2023. doi: 10.1038/s41598-023-29230-7.

Christopher C. Steel, John W. Blackman, and Leigh M. Schmidtke. Grapevine bunch rots: Impacts on wine composition, quality, and potential procedures for the removal of wine faults. *Journal of Agricultural and Food Chemistry*, 61(22):5189–5206, 2013. doi: 10.1021/jf400641r.

Márton Szabó, Anna Csikász-Krizsics, Terézia Dula, Eszter Farkas, Dóra Roznik, Pál Kozma, and Tamás Deák. Black rot of grapes (Guignardia bidwellii)–a comprehensive overview. *Horticulturae*, 9(2):130, 2023. doi: 10.3390/horticulturae9020130.

J. Tummers, A. Kassahun, and B. Tekinerdogan. Reference architecture design for farm management information systems: a multi-case study approach. *Precision Agriculture*, 22:22–50, 2021. doi: 10.1007/s11119-020-09728-0.

Joep Tummers, Ayalew Kassahun, and Bedir Tekinerdogan. Obstacles and features of farm management information systems: A systematic literature review. *Computers and Electronics in Agriculture*, 157:189–204, 2019. doi: 10.1016/j.compag.2018.12.044.

Havva Uyar, Ioannis Karvelas, Stamatia Rizou, and Spyros Fountas. Data value creation in agriculture: A review. *Computers and Electronics in Agriculture*, 227(Part 2):109602, 2024. doi: 10.1016/j.compag.2024.109602.

Eric Walling and Céline Vaneeckhaute. Developing successful environmental decision support systems: Challenges and best practices. *Journal of Environmental Management*, 264:110513, 2020. doi: 10.1016/j.jenvman.2020.110513.

Xiaoyue Xie, Yuan Ma, Bin Liu, Jinrong He, Shuqin Li, and Hongyan Wang. A deep-learning-based real-time detector for grape leaf diseases using improved convolutional neural networks. *Frontiers in Plant Science*, 11:751, 2020. doi: 10.3389/fpls.2020.00751.

Zhaoyu Zhai, José Fernán Martínez, Victoria Beltran, and Néstor Lucas Martínez. Decision support systems for agriculture 4.0: Survey and challenges. *Computers and Electronics in Agriculture*, 170:105256, 2020. doi: 10.1016/j.compag.2020.105256.

Zhiqiang Zhang, Yanjie Qiao, Yuhui Guo, and Dongjian He. Deep learning based automatic grape downy mildew detection. *Frontiers in Plant Science*, 13:872107, 2022. doi: 10.3389/fpls.2022.872107.